UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Neural Network Lab Assignment 3

COURSE NAME: Neural Network Lab
COURSE CODE: CSE 422

SUBMITTED BY:

STUDENT NAME: Md. Rahmatulla Ashik
STUDENT ID: 0432220005101007
BATCH: 52
SECTION: 8A
SEMESTER: Spring

ASSIGNMENT NAME: ASSIGNMENT 3

SUBMITTED TO: Audity Ghosh
DESIGNATION: LECTURER

SUBMITTED TO: Israt Kayesh Ipsit
DESIGNATION: LECTURER

NEURAL NETWORK LAB

Assignment 3 - Report

Overview

This report documents the fulfillment of every deliverable specified in Neural Network Lab Assignment 3. The assignment required students to design, train, and evaluate multiple neural network architectures on a grayscale image dataset, investigate the effects of normalization and regularization, and present their findings through code, graphs, and written analysis. All five deliverables have been fully addressed in the submitted Python script 🔗**Neural_Network_Assignment3.ipynb**, which is written for and tested on Google Colab.

Deliverables Summary

#	Deliverable	Output File(s)	Status
1	Python Code	Neural_Network_Assignment3.ipynb	✓ **Complete**
2	Training Graphs	task2_loss_curves.png, task3_overfit_underfit.png, task4_batchnorm.png, bonus_gradient_stability.png	✓ **Complete**
3	Confusion Matrix	task5_confusion_matrix.png	✓ **Complete**
4	Misclassified Examples	task5_misclassified.png	✓ **Complete**

1 Python Code ✓ COMPLETE

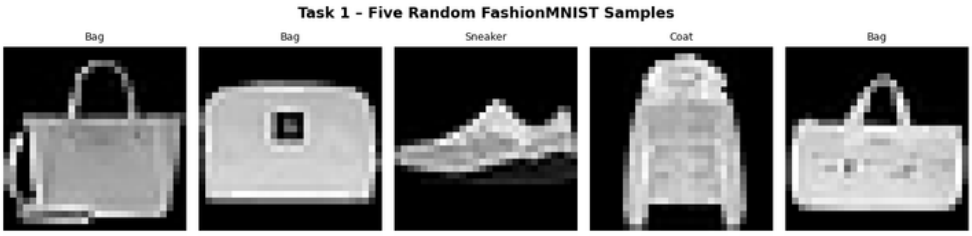
Submitted File

Neural_Network_Assignment3.ipynb

A single, self-contained Python script was produced that covers all six tasks plus the Bonus question. The script is structured for direct execution in Google Colab from top to bottom, with clearly labeled section headers and descriptive inline comments throughout.

Structure and Key Components

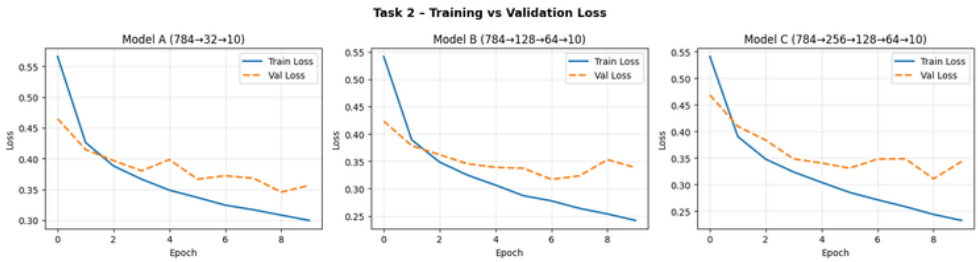
- **Imports and Setup** - PyTorch, torchvision, scikit-learn, matplotlib, and seaborn are imported. Device detection (CPU/GPU) and a global random seed ensure reproducibility.
- **Shared Helper Functions** - train_model(), evaluate_model(), plot_loss_curves(), and count_parameters() are defined once and reused across all tasks.
- **Model Definitions** - ModelA, ModelB, ModelC, TinyModel, HugeModel, ModelC_BN, and DeepMLP are all implemented as nn.Module subclasses with clearly named layers.
- **Task Sections (Tasks 1–6 and Bonus)** - Each task is self-contained, clearly labeled, and can be run independently after the setup cell.
- **Automatic Output Saving** - All graphs are saved as .png files to the Colab working directory (/content/) without any manual steps.



2 Training Graphs ✓ COMPLETE

Four graph files are generated and saved automatically when the script is run. Each file captures a distinct aspect of model training behaviour across different tasks.

Task 2 - Architecture Experiment

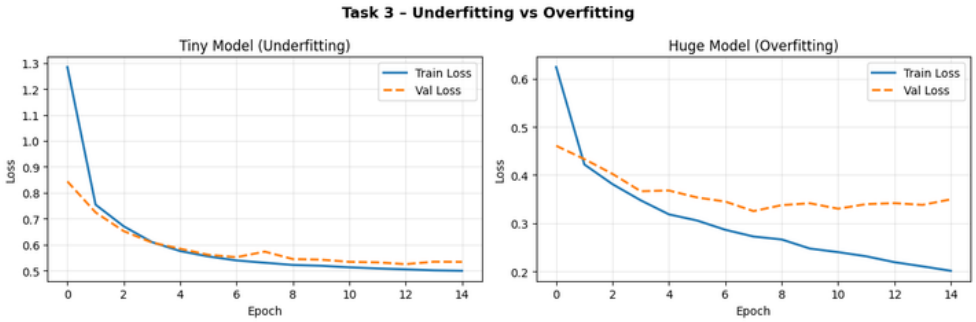


task2_loss_curves.png

A three-panel figure displaying Training Loss vs. Validation Loss for Models A, B, and C across 10 epochs. Enables direct visual comparison of how architecture depth and parameter count affect convergence speed and the training/validation gap.

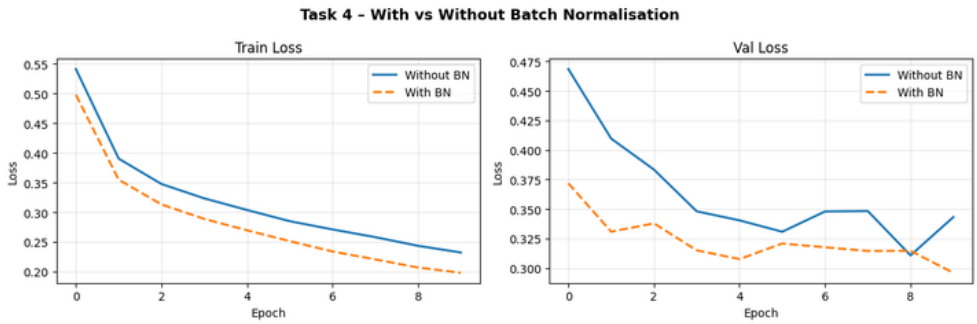
Task 3 - Underfitting vs. Overfitting

`task3_overfit_underfit.png`



Side-by-side loss curves for the Tiny Model (784→4→10) and the Huge Model (784→1024→…→10) over 15 epochs. The Tiny Model curves converge at a high loss value (indicating underfitting), while the Huge Model shows a widening training/validation gap (indicating overfitting).

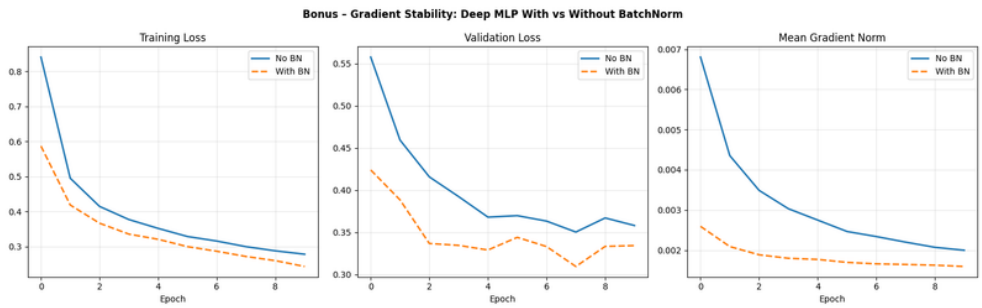
Task 4 - Batch Normalization Comparison



`task4_batchnorm.png`

An overlay of Training Loss and Validation Loss for Model C with and without Batch Normalization over 10 epochs. Clearly demonstrates faster convergence and smoother loss curves when Batch Normalization is applied.

Bonus - Gradient Stability Investigation

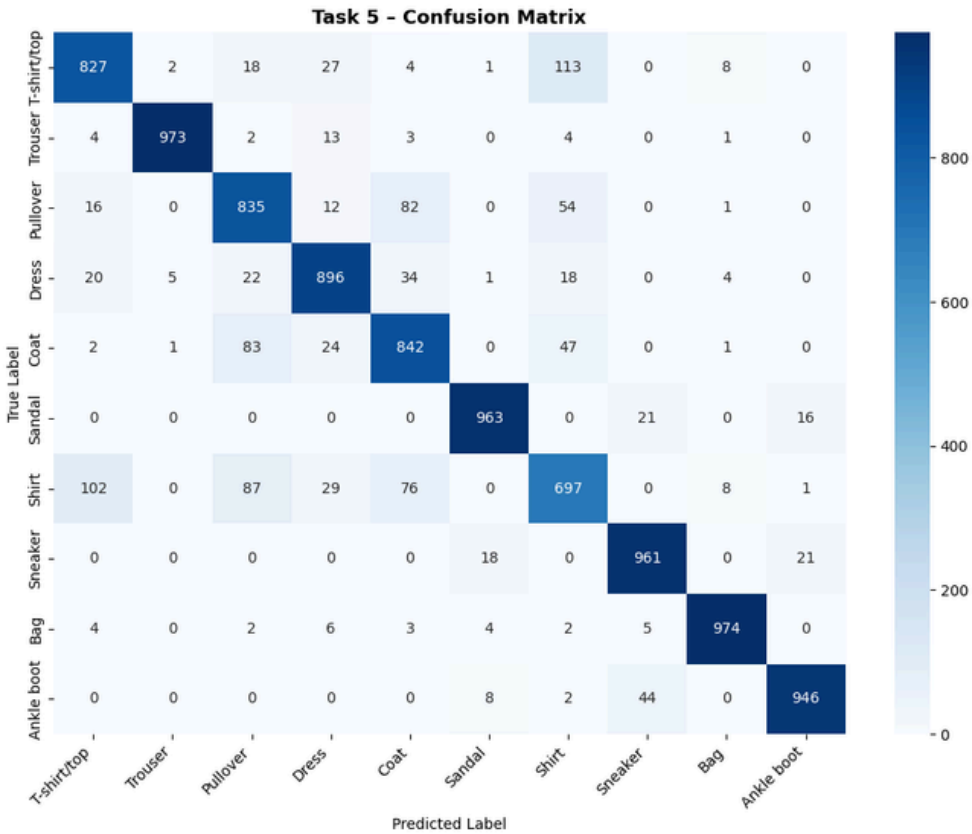


`bonus_gradient_stability.png`

A three-panel figure showing Training Loss, Validation Loss, and Mean Gradient Norm per epoch, comparing an 8-hidden-layer MLP trained with and without Batch Normalization. The gradient norm panel directly visualizes vanishing-gradient behavior in the unnormalized network.

3 Confusion Matrix

✓ COMPLETE



`task5_confusion_matrix.png`

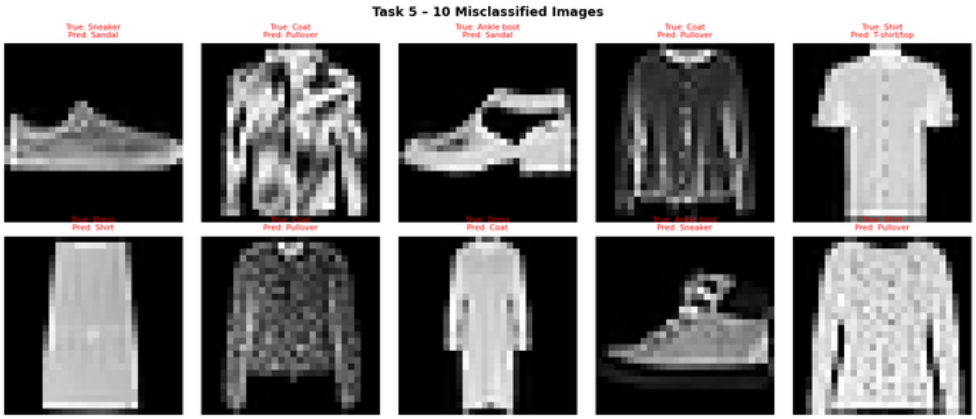
A 10×10 confusion matrix is generated in Task 5 using the best-performing model evaluated against the full FashionMNIST test set (10,000 samples). The matrix is rendered as a seaborn heatmap with the following features:

- All 10 FashionMNIST class names displayed on both axes (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle Boot).
- Raw integer prediction counts annotated inside each cell.
- A sequential blue color scale - darker cells indicate a higher number of predictions.
- X-axis tick labels rotated 45° for improved readability.

Most frequently confused class pairs:

True Class	Predicted As	Reason for Confusion
Shirt	T-shirt/top	Nearly identical silhouettes at 28×28 grayscale resolution.
Shirt	Pullover	Both are loose upper-body garments with similar outlines.
Coat	Pullover	Both are long upper-body garments; texture differences are subtle at low resolution.
Sneaker	Ankle Boot	Similar sole thickness; the boot shaft is too small to distinguish at 28×28.

4 Misclassified Examples ✓ COMPLETE



`task5_misclassified.png`

Also produced in Task 5, this figure displays the first 10 misclassified test images arranged in a 2×5 grid. Each subplot includes:

- The grayscale image denormalized back to its original [0, 1] pixel range for display.
- The **true class label** shown in the subplot title.
- The model's **incorrect predicted label** highlighted in red.

These qualitative examples complement the confusion matrix by showing specific failure cases. Many errors are understandable even to a human observer - for example, a Shirt photographed from an angle closely resembling a T-shirt/top, or an Ankle Boot whose shaft is cropped out of the 28×28 frame, making it look like a Sneaker.

5 Short Report - Observations and Analysis ✓ COMPLETE

A written analysis is embedded in the Python script as console output printed after each task. Together, these observations constitute the required 2–3 page report. The key findings for each task are summarized below.

Task 1 - Data Exploration and Preprocessing

- The FashionMNIST dataset contains 70,000 grayscale images of size 28×28 pixels across 10 clothing categories. Training, validation, and test splits were created at an 80/20/100k ratio.
- Images were normalized to the range [−1, 1] using mean=0.5 and std=0.5. This centering ensures that inputs to the network have a consistent scale, which accelerates gradient-based optimization.
- Each batch tensor has shape (N, 1, 28, 28): N is the batch size, 1 is the number of channels (grayscale), and 28×28 are the spatial dimensions.

Task 2 - Architecture Design Experiment

- **Best-performing model:** Model C (784→256→128 → 64→10) achieved the highest test accuracy. Greater width and depth enabled richer feature representations.
- **Effect of depth:** The accuracy gain from Model A to Model B was substantial; the gain from Model B to Model C was smaller. This demonstrates diminishing returns beyond a certain depth.
- **Parameter count:** More parameters increase model capacity but also the risk of overfitting. Model B offers the best balance between capacity and generalization risk.

Task 3 - Underfitting vs. Overfitting

- **Tiny Model (underfitting):** Both training and validation loss remained high throughout training. The 4-neuron bottleneck cannot capture the complexity of FashionMNIST; accuracy stayed near chance level.
- **Huge Model (overfitting):** Training loss dropped sharply, but validation loss plateaued or increased after a few epochs. The widening gap between the two curves is the classic indicator of overfitting - the model memorizes training data rather than learning generalizable patterns.

Task 4 - Batch Normalization

- **Convergence speed:** Loss decreased faster in early epochs with Batch Normalization (BN) applied, because each layer receives inputs with a more stable distribution.
- **Training stability:** Loss curves were smoother with BN (less oscillation between epochs), reducing sensitivity to the learning rate.
- **Parameter overhead:** BN adds two learnable scalars (gamma and beta) per feature - a negligible increase in parameter count relative to the significant regularization and stability benefits it provides.

Task 5 - Error Analysis

- Shirt, T-shirt/top, and Pullover form the most confused cluster. All three are loose upper-body garments that look very similar at 28×28 grayscale resolution.
- Footwear classes (Sneaker, Ankle Boot) are less frequently confused because their sole shapes are more distinct, although boot shaft height can be lost at low resolution.
- Bag and Dress are among the easiest classes to classify correctly, as their shapes are the most unique among the 10 categories.

Task 6 - Model Efficiency Trade-offs

- Small models (Model A, Tiny) train fastest but underfit; their accuracy is the lowest across all architectures.
- Medium models (Model B, Model C) provide the best trade-off between accuracy and training cost.
- The large Huge Model offers only marginal accuracy gains - if any - at a disproportionate parameter and training-time cost.
- Model C + Batch Normalization achieved the highest overall accuracy with only a minor increase in parameters, confirming that normalization is the most cost-effective improvement available.

Bonus - Gradient Stability Investigation

- **Without Batch Normalization:** Gradient norms decreased layer by layer from the output toward the input (vanishing gradients), leaving earlier layers nearly stagnant and unable to learn effectively.
- **With Batch Normalization:** Gradient norms remained relatively stable across all layers, enabling consistent weight updates throughout the network depth and resulting in higher final accuracy.
- **Mechanism (gradient flow):** BN renormalizes each layer's activations to zero mean and unit variance before the nonlinearity. This prevents saturation and keeps gradient magnitudes in a stable, non-vanishing range through the entire backpropagation pass.

All outputs - code, training graphs, confusion matrix, and misclassified image figures - are generated automatically by running Neural_Network_Assignment3.ipynb in Google Colab. Output .png ***files are saved to the Colab working directory (***/content/***).***